

فاز ۲ پروژه اصول طراحی کامپایلر

امیرعلی فدیشه ای

سروش صابری فر

4022262171 – 4022262159

مقدمه :

در این پروژه، تمرکز اصلی بر روی دو مرحله مهم از فرایند شبیه سازی کامپایلر قرار دارد:

- تحلیل لغوی یا Lexical Analysis
- تحلیل معنایی یا Semantic Analysis

در فاز اول، هدف شناسایی توکن های برنامه ورودی، تشخیص اجزای لغوی زبان و ساخت جدول نماد اولیه است. در فاز دوم، با استفاده از جدول نماد ساخته شده، خطاهای معنایی برنامه شناسایی و گزارش می شوند. بر اساس توضیحات فاز اول پروژه، تحلیلگر لغوی باید بتواند انواع مختلف توکن ها را شناسایی کند. این توکن ها شامل شناسه ها، کلمات کلیدی، نوع های داده، عملگرها، جداکننده ها، اعداد، رشته ها، کاراکترها، مقادیر Boolean و مقدار null هستند. همچنین در این فاز، جدول نماد برای شناسه ها نیز ساخته می شود. در فاز دوم، هدف استفاده از جدول نماد برای تشخیص خطاهای معنایی است. این خطاها شامل تعریف تکراری متغیر در یک Scope، استفاده از متغیر تعریف نشده، ناسازگاری نوع ها، خطاهای مربوط به آرایه، خطاهای مربوط به ارث بری و سایر موارد معنایی هستند.

گرامر زبان این پروژه در فایل javaMinusMinus2.g4 تعریف شده است. این فایل مبنای تولید Lexer و Parser توسط ANTLR است و قوانین لغوی و نحوی زبان را مشخص می کند.

ساختار کلی پروژه

ساختار پروژه به صورت منظم در چند پوشه اصلی سازماندهی شده است. بر اساس ساختار پروژه، پوشه ها و فایل های اصلی به شکل زیر هستند:

- پوشه docs

این پوشه برای نگهداری فایل مستندات و گزارش نهایی پروژه استفاده می شود.

- پوشه lib

این پوشه شامل کتابخانه های خارجی مورد نیاز پروژه است. فایل موجود در این پوشه، فایل مربوط به ANTLR است:

antlr-4.13.2-complete.jar

این فایل برای تولید و اجرای بخش های مرتبط با Lexer و Parser استفاده می شود.

- بخش src

بخش src هسته اصلی پروژه را تشکیل می دهد و شامل فایل های اصلی پیاده سازی کامپایلر است. فایل های مهم این بخش عبارت اند از:

javaMinusMinus2.g4

Main.java

CompleteSymbolTableBuilder.java

SymbolTable.java

SymbolInfo.java

SemanticAnalyzer.java

TypeChecker.java

این فایل ها منطق اصلی تحلیل لغوی، نحوی، ساخت جدول نماد و تحلیل معنایی را تشکیل می دهند.

- بخش test

این بخش شامل فایل های تست پروژه است:

test_correct.txt

test_errors.txt

فایل اول شامل نمونه برنامه های صحیح و فایل دوم شامل نمونه برنامه های دارای خطا است.

- فایل های متفرقه

در ریشه پروژه فایل هایی مانند موارد زیر قرار دارند:

run.bat

cleaner.bat

فایل run.bat برای اجرای پروژه و فایل cleaner.bat برای پاک سازی خروجی های کامپایل استفاده می شود.

• پوشه pic

محتویات این پوشه شامل تصاویر ورودی ها تست و خروجی های متناظر و همچنین پوشه بندی پروژه برای درک بهتر پروژه می باشد.

معماری کلی سیستم

معماری پروژه به صورت مرحله ای طراحی شده است. هر مرحله خروجی خود را در اختیار مرحله بعدی قرار می دهد. جریان کلی اجرای کامپایلر به این صورت است:

1. دریافت فایل ورودی
2. خواندن محتوای فایل
3. اجرای Lexer برای تولید توکن ها
4. اجرای Parser برای تولید درخت تجزیه
5. ساخت جدول نماد با پیمایش درخت تجزیه
6. اجرای تحلیل معنایی
7. بررسی نوع عبارت ها و دستورات
8. جمع آوری خطاها
9. نمایش نتیجه نهایی

در این معماری، کلاس Main.java نقش هماهنگ کننده اصلی را دارد. این کلاس Lexer و Parser را اجرا می کند، سپس ساخت جدول نماد را انجام می دهد و در ادامه تحلیل معنایی را فراخوانی می کند.

توضیح نقش فایل های اصلی پروژه

فایل Main.java

فایل Main.java نقطه ورود اصلی برنامه است. این فایل وظیفه هماهنگ کردن مراحل مختلف اجرای کامپایلر را بر عهده دارد. وظایف اصلی این فایل عبارت اند از:

- دریافت مسیر فایل ورودی
- خواندن برنامه ورودی
- ایجاد Lexer و Parser
- تولید درخت تجزیه
- اجرای سازنده جدول نماد
- اجرای تحلیلگر معنایی
- جمع آوری و نمایش خطاها

این فایل خود مستقیماً تمام بررسی های معنایی را انجام نمی دهد، بلکه اجرای کلاس های تخصصی دیگر را مدیریت می کند.

فایل SymbolInfo.java

فایل SymbolInfo.java برای نگهداری اطلاعات مربوط به هر نماد در جدول نماد استفاده می شود. هر نماد می تواند یک کلاس، متد، متغیر، پارامتر یا عضو دیگری از برنامه باشد. اطلاعات نگهداری شده در این کلاس شامل موارد زیر است:

- نوع نماد
- نام نماد
- نوع داده ای نماد
- Scope تعریف
- سطح دسترسی
- وضعیت static ، final یا abstract بودن
- پارامترهای متد
- نوع بازگشتی متد
- کلاس والد در صورت وجود وراثت

این اطلاعات برای تحلیل معنایی ضروری هستند. برای مثال، برای بررسی نوع متغیرها، تشخیص متدها، بررسی ارث بری و اعتبارسنجی استفاده از شناسه ها، اطلاعات این کلاس مورد استفاده قرار می گیرد.

فایل SymbolTable.java

فایل SymbolTable.java مسئول مدیریت جدول نماد و Scope ها است. این کلاس ساختار اصلی ذخیره سازی و جستجوی نمادها را فراهم می کند. وظایف اصلی این فایل عبارت اند از:

- ایجاد Scope سراسری

- مدیریت Scope فعلی

- ورود به Scope جدید

- خروج از Scope فعلی

- افزودن نماد جدید

- تشخیص نماد تکراری در یک Scope

- جستجوی نماد در Scope های فعال

- چاپ اطلاعات جدول نماد

جدول نماد یکی از اجزای کلیدی پروژه است؛ زیرا تحلیلگر معنایی برای تشخیص بسیاری از خطاها به آن وابسته است.

فایل CompleteSymbolTableBuilder.java

فایل CompleteSymbolTableBuilder.java وظیفه ساخت کامل جدول نماد را بر عهده دارد. این کلاس به عنوان Listener عمل می کند و هنگام پیمایش درخت تجزیه، اطلاعات لازم را وارد جدول نماد می کند. وظایف اصلی این فایل عبارت اند از:

- تشخیص ورود به کلاس ها

- ثبت اطلاعات کلاس ها

- مدیریت وراثت

- تشخیص ورود به متدها

- ثبت اطلاعات متدها

- ثبت پارامترهای متد
- ثبت متغیرها
- مدیریت Scope کلاس و متد
- تشخیص برخی خطاهای اولیه مانند تعریف تکراری

این فایل ارتباط مستقیم میان ساختار نحوی برنامه و جدول نماد را برقرار می کند.

فایل SemanticAnalyzer.java

فایل SemanticAnalyzer.java تحلیلگر معنایی اصلی پروژه است. این کلاس بعد از ساخته شدن جدول نماد اجرا می شود و با استفاده از اطلاعات موجود در جدول نماد، صحت معنایی برنامه را بررسی می کند. وظایف اصلی این فایل عبارت اند از:

- پیمایش درخت تجزیه
- بررسی شرط های دستورات if
- بررسی شرط های حلقه while
- بررسی شرط های حلقه for
- بررسی سازگاری نوع در assignment
- بررسی سازگاری نوع در مقداردهی اولیه متغیرها
- استفاده از Type Checker برای تعیین نوع عبارت ها
- ثبت و گزارش خطاهای معنایی

یکی از مهم ترین وظایف این کلاس، بررسی Boolean بودن شرط ها است. در زبان های مشابه Java، شرط های مربوط به if، while و for باید از نوع Boolean باشند. اگر عبارتی از نوع دیگر در جایگاه شرط استفاده شود، خطای معنایی رخ می دهد.

فایل TypeChecker.java

فایل TypeChecker.java یک کلاس تخصصی برای بررسی نوع ها است. این کلاس توسط Semantic Analyzer استفاده می شود و وظیفه دارد نوع عبارت ها را تعیین کرده و سازگاری نوعی را بررسی کند. وظایف اصلی این فایل عبارت اند از:

- تعیین نوع عبارت ها
- بررسی سازگاری دو نوع
- بررسی استفاده صحیح از عملگرهای ریاضی
- بررسی استفاده صحیح از عملگرهای منطقی
- بررسی استفاده صحیح از عملگرهای رابطه ای
- بررسی استفاده صحیح از عملگرهای برابری
- بررسی اندیس آرایه
- بررسی اعمال شدن دسترسی آرایه ای روی نوع آرایه
- بررسی استفاده صحیح از ویژگی length
- تشخیص متغیر تعریف نشده
- تشخیص استفاده از متغیر قبل از مقداردهی
- بررسی حدود آرایه در موارد قابل تشخیص

این کلاس بخش مهمی از تحلیل معنایی را انجام می دهد و نقش آن در تشخیص خطاهای نوعی بسیار مهم است.

جدول نماد و مدیریت Scope ها

جدول نماد ساختاری است که اطلاعات شناسه های برنامه را ذخیره می کند. هر شناسه ممکن است مربوط به کلاس، متد، متغیر، پارامتر یا سایر عناصر زبان باشد. در این پروژه، جدول نماد علاوه بر نگهداری نمادها، وظیفه مدیریت Scope ها را نیز بر عهده دارد.

مفهوم Scope

Scope محدوده ای از برنامه است که در آن یک شناسه معتبر است. برای مثال، متغیری که داخل یک متد تعریف می شود، معمولاً فقط در همان متد قابل دسترسی است. همچنین پارامترهای یک متد فقط در محدوده همان متد اعتبار دارند.

مدیریت Scope ها

در پروژه، برای مدیریت Scope ها از ساختار پشته ای استفاده شده است. هنگام ورود به یک محدوده جدید، یک Scope جدید ایجاد می شود و هنگام خروج از آن محدوده، Scope فعلی بسته می شود. این روش باعث می شود که جستجوی نمادها مطابق با قواعد محدوده بندی زبان انجام شود.

تشخیص تعریف تکراری

یکی از خطاهای مهمی که با استفاده از جدول نماد قابل تشخیص است، تعریف دو نماد هم نام در یک Scope است. اگر متغیری در یک Scope تعریف شده باشد و دوباره نمادی با همان نام در همان Scope تعریف شود، خطای معنایی رخ می دهد.

تحلیل معنایی و بررسی نوع ها

تحلیل معنایی در این پروژه با استفاده از دو بخش اصلی انجام می شود:

Semantic Analyzer

Type Checker

بررسی شرط ها

در دستورات شرطی و حلقه ها، عبارت شرط باید از نوع Boolean باشد. بنابراین اگر در شرط if ، while یا for عبارتی از نوع int ، String یا هر نوع غیر Boolean استفاده شود، تحلیلگر معنایی خطا گزارش می دهد.

بررسی تخصیص مقدار

در assignment ، نوع سمت راست باید با نوع متغیر سمت چپ سازگار باشد. برای مثال، نمی توان مقدار رشته ای را به متغیری از نوع عددی نسبت داد. این بررسی توسط تحلیلگر معنایی و با کمک بررسی کننده نوع انجام می شود.

بررسی آرایه ها

یکی دیگر از بخش های مهم تحلیل معنایی، بررسی استفاده صحیح از آرایه ها است. در این پروژه موارد زیر بررسی می شوند:

- اندیس آرایه باید از نوع عدد صحیح باشد.
- روی متغیری که آرایه نیست نباید عمل دسترسی آرایه ای انجام شود.
- در صورت امکان، خارج بودن اندیس از محدوده آرایه تشخیص داده می شود.
- استفاده از ویژگی هایی مانند length باید فقط برای آرایه ها معتبر باشد.

بررسی عملگرها

در بررسی نوع عبارت ها، استفاده صحیح از عملگرها نیز ارزیابی می شود. برای مثال:

- عملگرهای ریاضی باید روی مقادیر عددی اعمال شوند.
- عملگرهای منطقی باید روی مقادیر Boolean اعمال شوند.
- عملگرهای رابطه ای باید با نوع های مناسب استفاده شوند.
- عملگرهای برابری باید روی نوع های سازگار اعمال شوند.

این بررسی ها توسط Type Checker انجام می شود.

فایل های تست پروژه

برای بررسی صحت عملکرد پروژه، دو فایل تست در نظر گرفته شده است.

test_correct.txt

این فایل شامل نمونه برنامه ای است که از نظر گرامری و معنایی صحیح هستند. هدف از این فایل، اطمینان از عملکرد صحیح کامپایلر روی ورودی های معتبر است. در این فایل نمونه هایی مانند تعریف کلاس اصلی، تعریف متد main و تعریف کلاس انتزاعی وجود دارد.

test_errors.txt

این فایل شامل برنامه هایی است که به صورت عمدی دارای خطا هستند. هدف از این فایل، بررسی توانایی کامپایلر در تشخیص و گزارش خطاهای مختلف است. نمونه خطاهای موجود در این فایل شامل موارد زیر است:

- استفاده از متغیر تعریف نشده
- عدم تطابق نوع در مقداردهی
- وجود دور در ارث بری
- استفاده از اندیس نامعتبر برای آرایه
- استفاده از مقدار رشته ای به عنوان اندیس آرایه

نحوه اجرای پروژه

برای اجرای پروژه، فایل run.bat در نظر گرفته شده است. این فایل یک اسکریپت اجرایی برای محیط ویندوز است. وظایف اصلی این اسکریپت عبارت اند از:

1. کامپایل فایل اصلی پروژه

2. اجرای کلاس Main

3. ارسال فایل ورودی به برنامه از طریق آرگومان خط فرمان

در این پروژه، فایل ورودی به عنوان آرگومان به برنامه داده می شود. سپس کلاس Main فرایند تحلیل را روی آن فایل انجام می دهد.

نحوه پاک سازی فایل های خروجی

برای حذف فایل های تولید شده در فرایند کامپایل، فایل cleaner.bat در نظر گرفته شده است. این فایل خروجی های میانی و فایل های تولید شده را حذف می کند. وظایف اصلی این اسکریپت عبارت اند از:

1. حذف فایل های کامپایل شده Java با پسوند class.

2. حذف فایل های توکن با پسوند tokens.